ENDTERM PROJECT ASSIGNMENT

Continuation of the Midterm Project: Resilient University Information System

Course: Fault Tolerance and Dependable Computing

1. Project Overview

This assignment continues the Midterm Project, “Fault-Tolerant University Information System.” Teams must extend their existing implementation rather than start a completely unrelated system. The endterm focuses on cloud computing and microservices resilience; fault detection, monitoring, and observability; security, resilience, and intrusion tolerance; and verification, validation, and fault-injection testing.

The goal is to show, through implementation and measured experiments, how the system detects abnormal behavior, limits the impact of failures or attacks, and recovers while preserving service availability and data integrity.

2. Learning Objectives

- Design a cloud-ready microservices architecture and explain its resilience properties.
- Instrument services with health checks, structured logs, metrics, and distributed traces or correlation IDs.
- Detect service failures and abnormal behavior using observable evidence and defined alert thresholds.
- Apply security controls and demonstrate how the system responds to selected attack or misuse scenarios.
- Validate system behavior through systematic tests and controlled fault injection.
- Compare endterm results with the midterm baseline and justify conclusions using measured evidence.

3. Required Topic Coverage

Course topic	Required focus	Expected evidence
Cloud Computing and Microservices Resilience	Service decomposition; API gateway; service discovery or configuration; replicas; health/readiness checks; load balancing; timeout/retry/circuit-breaker policies; graceful degradation; deployment or scaling strategy.	Updated architecture diagram, configuration, and a demonstration of at least one service or instance failure.
Fault Detection, Monitoring, and Observability	Structured logs, service-level metrics, health endpoints, correlation/request IDs, dashboards, alert rules, and identification of detection and recovery times.	Dashboard or equivalent visual report; sample logs/metrics; explanation of how an incident is detected and diagnosed.
Security, Resilience, and Intrusion Tolerance	Authentication and authorization; input validation; secrets/configuration handling; rate limiting or abuse protection; audit logging; safe failure behavior; least-privilege principles.	Threat model, implemented controls, and controlled tests of at least two security or misuse scenarios.
Verification, Validation, and Fault Injection Testing	Unit/integration/system tests; explicit acceptance criteria; repeatable fault injection; workload generation; data-integrity checks; baseline-versus-resilient comparison.	Test plan, scripts or commands, raw results/logs, and a results table with pass/fail outcomes.

4. Scope and Continuity Requirements

- Reuse the university services and core business workflow developed for the midterm project. Teams may refine the architecture but must document major changes.
- Keep at least three independently identifiable services or modules, including a data-storage component.
- Retain a baseline or midterm configuration so that results can be compared fairly.

- Do not claim that a mechanism works solely because it appears in a diagram: demonstrate its behavior and record evidence.
- Use synthetic or anonymized data only. Do not test against real users, production systems, or systems for which the team lacks authorization.

5. Implementation Requirements

1. Cloud/microservices resilience: containerize the application where practical; configure health checks and a documented restart/recovery or replica strategy; implement appropriate timeouts and bounded retries; and demonstrate graceful degradation for at least one non-critical feature.
2. Observability: expose or collect at least three useful metrics (for example, request count, error rate, and latency); produce structured logs; and correlate requests across service boundaries using a request/correlation ID or equivalent.
3. Incident detection: define at least three alert conditions or threshold rules. For each, state what it detects, the threshold, and the response expected from the team or system.
4. Security and intrusion tolerance: implement at least three applicable controls from authentication/authorization, input validation, rate limiting, secure secrets handling, audit logging, dependency/configuration checks, or duplicate/abuse request protection.
5. Testing: implement automated unit or integration tests for critical workflows and create repeatable scripts or documented steps for fault-injection experiments.

6. Required Experiments

Conduct at least six experiments. At least four must involve fault injection or abnormal conditions, and at least two must evaluate security or misuse handling. Restore the system to a known state between experiments.

Experiment	Example scenario	Evidence to record
Service/instance failure	Stop one service instance or terminate a container.	Detection time, recovery time, failed/successful requests, user-visible impact.
Dependency timeout	Delay or block calls to the database or another service.	Timeout behavior, retries, error rate, latency, recovery.
Load spike	Generate a documented increase in concurrent requests.	Throughput, latency percentiles if available, error rate, resource use.
Monitoring/alert test	Trigger a known threshold or simulated incident.	Metric/log evidence, whether the alert condition is met, detection delay.
Unauthorized access	Attempt a protected operation without valid authorization.	Expected denial, audit event, and confirmation that data was not exposed or changed.
Abusive or malformed requests	Send invalid input or a burst of requests to a test endpoint.	Validation/rate-limit behavior, service stability, logs, and data integrity.
Recovery and consistency	Interrupt a payment or academic-record operation in a controlled test.	Duplicate/lost records, consistency checks, and recovery outcome.

For every experiment, report the hypothesis, environment/configuration, steps, expected result, observed result, number of repetitions, and limitations. Never fabricate measurements; label simulated or illustrative data clearly.

7. Required Measurements

- Request success and failure counts; error rate; and response latency (average and, where possible, p95).
- Incident detection time and service recovery time.
- Availability or successful-request ratio over the defined test interval.
- Number of alerts triggered and whether each alert was correct for the test scenario.
- Data-integrity outcome, including checks for missing or duplicate transactions/records.
- Comparison between the midterm/baseline configuration and the endterm implementation.

8. Report Structure

- Introduction and relationship to the midterm project
- Updated requirements, assumptions, and threat model
- Cloud/microservices architecture and resilience design
- Monitoring, logging, metrics, and alerting design
- Security controls and intrusion-tolerance strategy
- Verification and validation plan
- Fault-injection and security-test methodology
- Results and baseline-versus-endterm comparison
- Discussion, limitations, and lessons learned
- Conclusion and references
- Appendix: source code links, configurations, test scripts, logs, and additional results

9. Deliverables

- Updated source-code repository with run instructions
- Updated architecture diagram and deployment/configuration files
- Monitoring dashboard or equivalent evidence, sample logs, metrics, and alert definitions
- Threat model and description of implemented security controls
- Automated tests and repeatable fault-injection/test scripts
- Experiment results table and final technical report
- Live or recorded demonstration of at least three scenarios, including one service failure and one security/misuse test

10. Assessment Rubric

Criterion	Weight	Assessment focus
Cloud and microservices resilience	20%	Architecture, deployment, failure containment, recovery, and graceful degradation.
Monitoring and observability	20%	Useful metrics/logs, request correlation, dashboards/alerts, and incident diagnosis.
Security and intrusion tolerance	20%	Relevant threat model, correctly implemented controls, and demonstrated safe behavior.
Verification and fault-injection experiments	25%	Repeatability, quality of tests, measured results, data-integrity checks, and fair comparison.
Report and technical documentation	10%	Clarity, traceability, reproducibility, and honest discussion of limitations.
Demonstration and individual contribution	5%	Clear demonstration and ability to explain implementation decisions.

11. Recommended Technology

Teams may continue with the midterm stack (for example, Python/FastAPI, Docker, PostgreSQL or SQLite, and Git). Prometheus/Grafana or equivalent tools may be used for metrics and dashboards. Equivalent technologies are acceptable if the team explains how they satisfy the requirements.

12. Success Criteria

The project is successful when the team demonstrates that the extended system can detect and diagnose selected incidents, limit the impact of service failures and tested misuse, recover predictably, and preserve data

integrity. Conclusions must be supported by reproducible evidence and compared with the midterm implementation.